

UNIVERSIDAD DE GRANADA

PRÁCTICA 2:

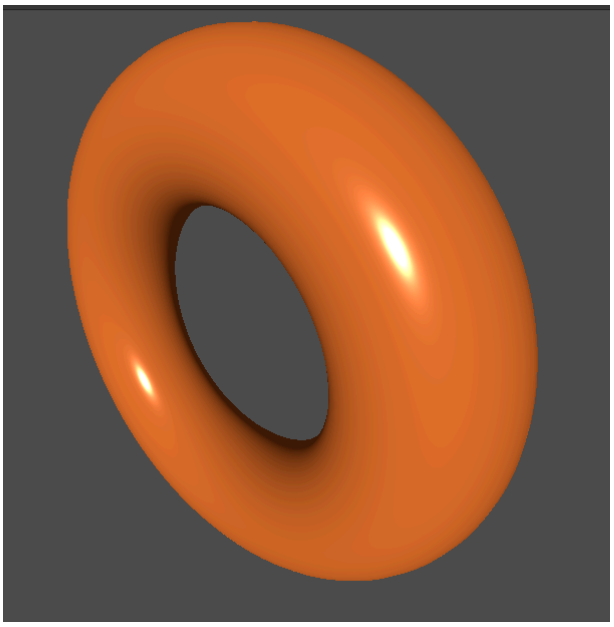
Memoria de la realización de los scripts de la práctica

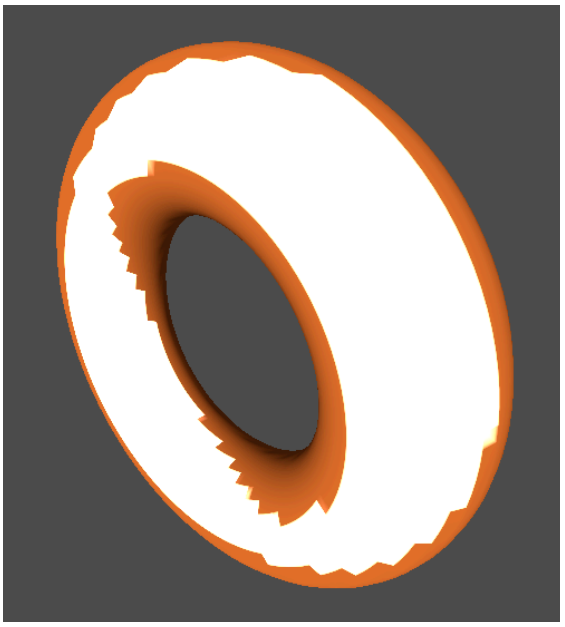
ADRIÁN VARELA JIMÉNEZ

PROYECTO COMPLETO

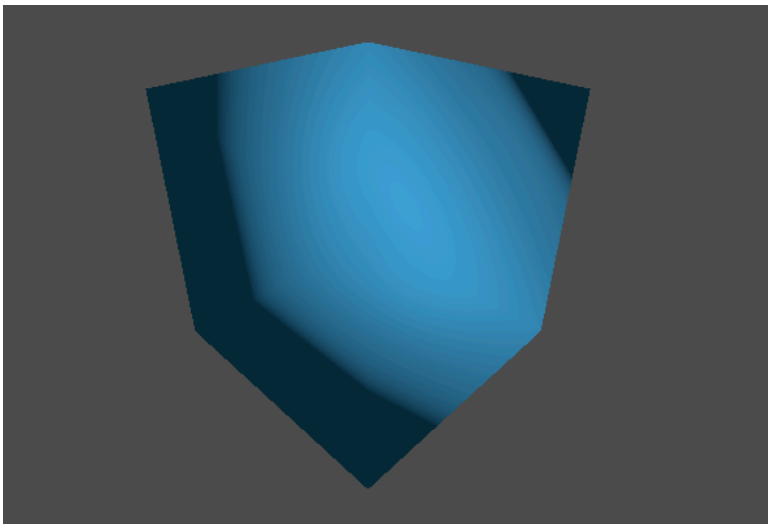
En esta memoria se va a detallar la realización de los scripts que he desarrollado para la realización de la práctica.

- Script **donut.gd**
  En este script lo único que hemos tenido que modificar ha sido la línea en la que se declara el tipo de sombreado que queremos realizar con el donut, cambiando así la primera línea que hay a continuación por la segunda:
  -Antes: mat.shading_mode=BaseMaterial3D.SHADING_MODE_PER_PIXEL
  -Despés:mat.shading_mode=BaseMaterial3D.SHADING_MODE_PER_VERTEX
  En mi caso lo que he hecho ha sido dejar la primera línea comentada y añadir la otra abajo para poder cambiar de modo de sombreado sin estar teniendo que reescribirla.
  Nuestro sombreado ha dejado de verse tan realista porque al ponerlo por vértice, se calcula la iluminación de un vértice y se mezcla en el triángulo con la de los otros vértices, dando así este aspecto tan poco realista.

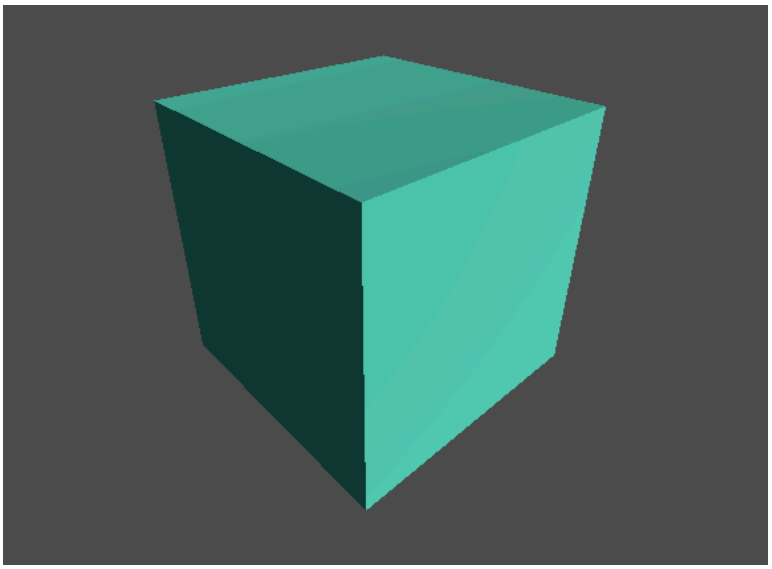




- Script **cubo_8.gd**
  Para hacer el cubo la lógica que hemos seguido ha sido utilizar sus 8 vértices reales, definiendo una matriz con únicamente estos 8 puntos. Así podemos tejer la malla de las 6 caras del cubo, compuestas por 2 triángulos cada una, por lo que realmente estamos tejiendo 12 triángulos.
  El problema de este cubo es que el renderizado con iluminación no queda nada realista, pareciendo como si la luz atravesase el cubo, haciendo una forma muy rara.



- Script **cubo_24.gd**
  En este script implementamos la solución al fallo que hemos tenido en el cubo de 8 vértices que hemos hecho anteriormente, generando cada una de las 6 caras de una forma independiente, teniendo así 24 vértices aunque sigamos teniendo los mismos 12 triángulos que antes.



- Script global **generador_mallas.gd**
  En este script hemos implementado la función **generarRevolución** la cual vamos a llamarla cuando queramos utilizarla para generar cualquier tipo de objeto por revolución.
  Este algoritmo utiliza un bucle con senos y cosenos a través del cual vamos rotando una serie de puntos para conseguir la figura deseada. Llevándolo a cabo, me he dado cuenta que si daba una vuelta hasta el copias + 1, la última

columna de vértices dejaba una marca en otro color debido a la renderización que tenía, por lo que he quitado el +1 y he utilizado el módulo al calcular la columna vecina: (s + 1) % copias, siendo así que en la última repetición del bucle, va a unir su columna con la primera de todas en vez de a una nueva copia creada.

Además, para evitar la duplicación de los vértices sobre los que rota la figura y la creación de triángulos degenerados, he implementado dos condiciones para cada caso:

-Duplicación de vértices: con los dos primeros if que hay, tanto para vértice superior como para el inferior, si x == 0 (estamos en el centro), forzamos que las esquinas compartan el mismo vértice central, omitiendo así la creación de vértices innecesarios.

-Triángulos degenerados: con los dos últimos if que hay, tanto para base superior como para la inferior, si x != 0 (no tienen anchura), no creamos un triángulo, porque al crear una línea recta y luego intentar calcular la normal, nos puede dar error.

- Script **revolucion.gd**

  En este script es en el que ponemos a prueba nuestro algoritmo de creación de objetos por resolución, para el que yo he usado una especie de forma de jarrón, y 128 rotaciones para que se vea una superficie uniforme.

  Una vez tenemos todo preparado, lo único que hay que hacer es llamar a nuestra función para que nos cree el objeto y crearnos nuestro objeto.

